EXP : 01

# Bresenham's line drawing algorithm (|m|<1)

AIM :-

To implement Bresenham's line drawing algo(|m|<1) in Python for drawing a line when the slope

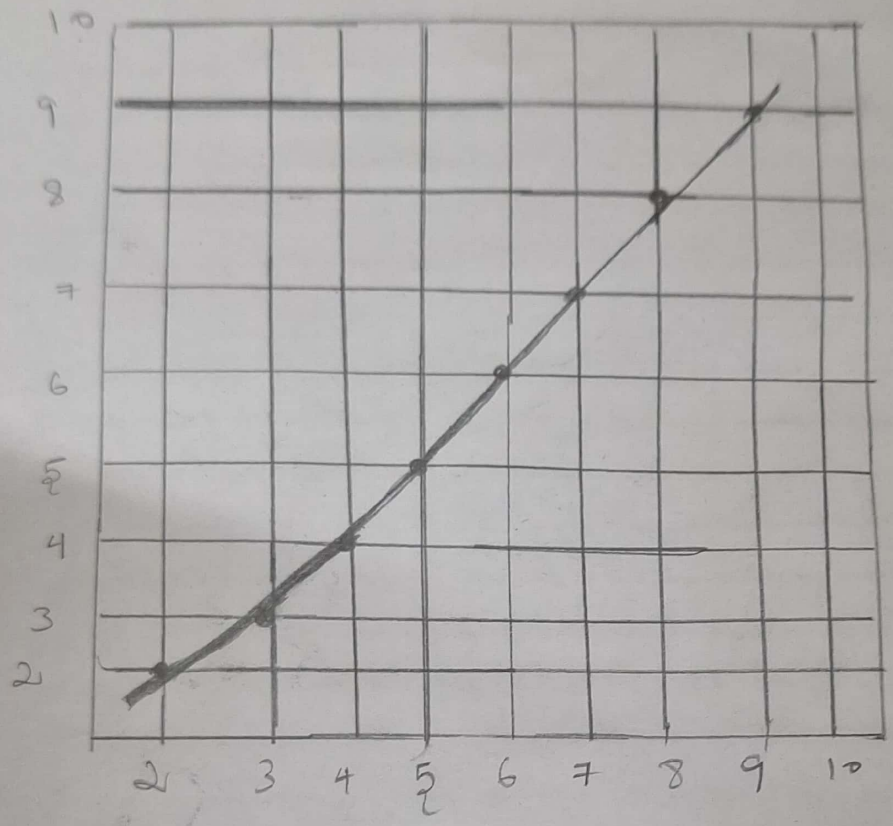
Code :-

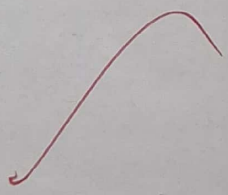
```
import matplotlib.pyplot as plt
def bresenham(x1, y1, x2, y2)
dx = x2 - x1
dy = y2 - y1
P = 2* dy - dx
x = x1
y = y1
x - Points =[x]
y - Points =[y]
while x1 < x2 :
    x += 1
    if P<0:
        P = P+2*dy
    else:
        y += 1
        P = P+2*dy - 2*dx
    x - Points.append(x)
    y - Points.append(y)
```

output :

x : [2, 3, 4, 5, 6, 7, 8, 9, 10]

y : [2, 3, 4, 5, 6, 7, 8, 9, 10]



Scanned with OKEN Scanner

```
plt.plot(x, y, 'bo')
plt.title("Bresenham line drawing")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.show()
```

# example

$x_1, y_1 = 2, 3$

$x_2, y_2 = 10, 7$

bresenham$(x_1, y_1, x_2, y_2)$

Result -

The Program was executed successfully Bresenham's line drawing algo for the input where slope $|m| < 1$.

EXP 10: Bresenham's circle, line ellipse algorithm

AIM:

To implement Bresenham's circle, line, ellipse in python.

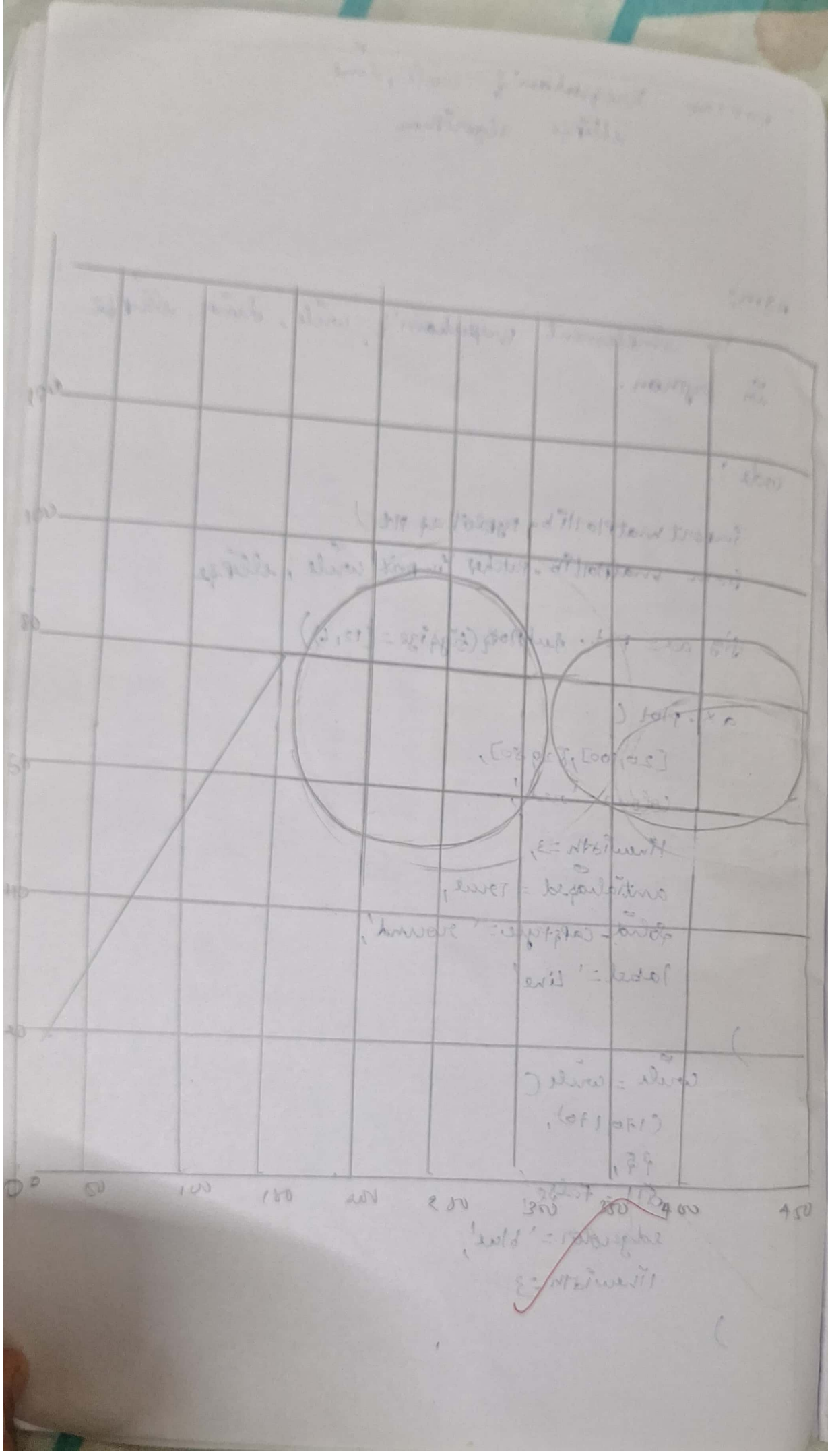
Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import circle, ellipse

fig, ax = plt.subplots(figsize=(12,6))

ax.plot(
    [20,100],[20,80],
    colour='red',
    linewidth=3,
    antialiased=True,
    solid_capstyle='round',
    label='Line'
)

circle = circle(
    (170,170),
    75,
    fill=False,
    edgecolor='blue',
    linewidth=3
)
```



```
ax.add_patch(ellipse)

ax.set_xlim(0,360)
ax.set_ylim(0,130)

ax.set_aspect('equal')
ax.grid(True)

ax.set_title("Line, circle and ellipse")
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")

ax.legend()

plt.show()
```

Result:

Hence we got and achieved the Bresenham's circle, line, ellipse drawing algorithm

Scanned with OKEN Scanner

Experiment 8 DDA line drawing algorithm.

AIM:-

To implement the DDA line drawing Algorithm

code:

```
import matplotlib.pyplot as plt

def dda(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1

    steps = int(max(abs(dx), abs(dy)))

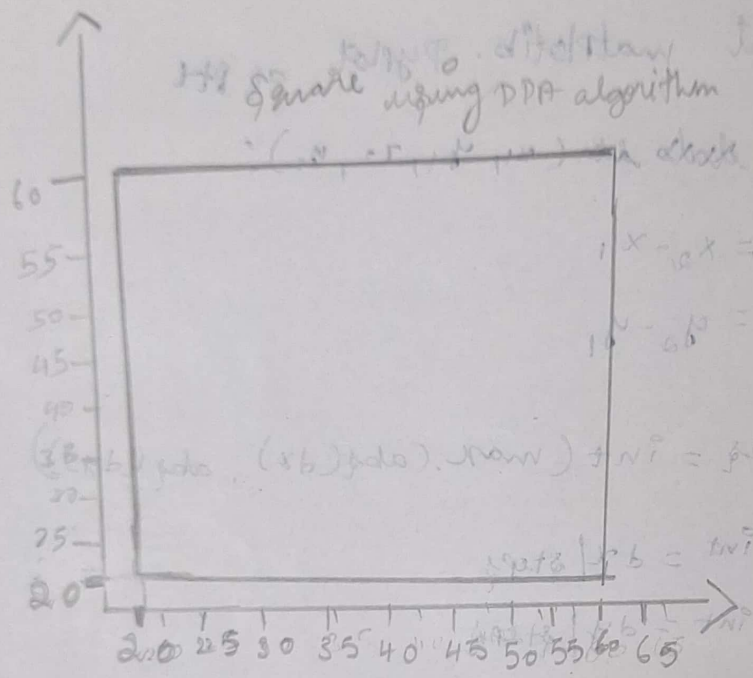
    x_inc = dx / steps
    y_inc = dy / steps

    x = x1
    y = y1

    x_points = []
    y_points = []

    for i in range(steps + 1):
        x_points.append(round(x))
        y_points.append(round(y))
```

output:



```
x+1 = x+inc
y+1 = y+inc

return x, y, point1, point2

x1, y1 = 2, 3
x2, y2 = 8, 6

x, y = dda(x1, y1, x2, y2)

plt.plot(x, y, marker='o')
plt.grid(True)
plt.title("DDA line drawing algo")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Result :-

Hence the program is verified successfully

EXP:04 BUILD AN OBJECT USING BRESENHAM'S ALGORITHM

AIM:-

To build an object using bresenham algorithm

Code:-

```
import matplotlib.pyplot as plt

def line(x1,y1,x2,y2):
    dx = abs(x2-x1)
    dy = abs(y2-y1)
    sx = 1 if x2>x1 else -1
    sy = 1 if y2>y1 else -1
    err = dx-dy
    x=[]
    y=[]
    while True:
        x.append(x1)
        y.append(y1)
        if x1==x2 and y1==y2:
            break
        e2=2*err
        if e2>-dy:
            err-=dy
            x0+=sx
        if e2<dx:
            err+=dx
            y0+=sy
```

Scanned with OKEN Scanner

```
return points

def draw_line(x0, y0, x1, y1)
    points = bresenham_line(x0, y0, x1, y1)
    x = [p[0] for p in points]
    y = [p[1] for p in points]
plt.plot(x, y, '-', colour = 'purple', line width=2,
                                    marker=5)

draw_line(2.5, 5.5, 3.5, 4.5)
draw_line(2.5, 4.5, 3.5, 5.5)
draw_line(6.5, 5.5, 7.5, 4.5)
draw_line(6.5, 4.5, 7.5, 5.5)

plt.title("house with windows & lines using
        Bresenham's line algorithm)
```

Results :-

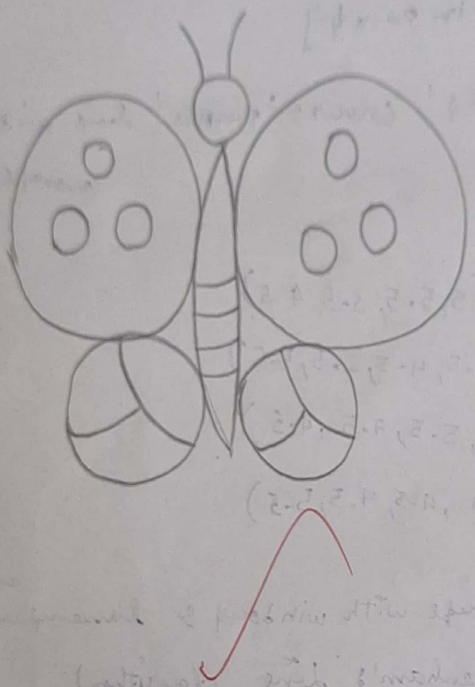
using bresenham's algorithm the rectangle was drawn successfully

# 1X-0f Butterfly using Bresenham's algorithm

AIM:- To draw a butterfly using bresenham's algorithm

code:-

```
import matplot.pyplot as PLT
import matplot.patches as Patches
fig, ax = PLT.subplots(figsize=(6,6))
deg add = ellipse (x, Y, width, height, color,
                   angle=0))
ellipse = Patches.ellipse((x,y): width, height, angle)
ax.add_patch(ellipse)
add ellipse (-0.9, 0.8, 1.4, 2.0, "skyblue")
add.ellipse(-0.8, 0.9, 0.4, 0.7, "red")
add.ellipse(0.0, 0.3, 0.2, "brown")
head = Patches.Circle((0,1,25), 0.2, color="black")
ax.add_patch(head)
ax.plot(-0.07, 1.3, "o", color="white")
ax.plot(0.07, 1.3, "o", color="white")
ax.plot([0.01, 1.3, "o"], (1.4, 1.8],
        color: black, line width=2]
```



Scanned with OKEN Scanner

```
ax.plot([0.05, 0.35], [1.4, 1.8],
        colour="black", linewidth=2)

ax.set_xlim(-2,2)
ax.set_ylim(-2,2)
ax.set_aspect("equal")
ax.axis("off")
plt.show()
```

Result:

using bresenham's algorithm the butterfly was drawn successfully

Ex:06 Draw a smiling face using bresenham's algorithm

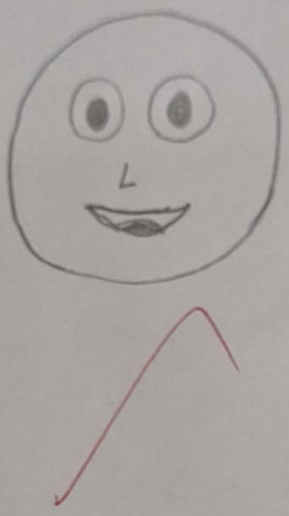
AIM:-

To draw a smiling face using bresenham's algorithm

Code:-

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(4,4))
ax.add_patch(plt.Circle((0,0), 1.0,
        colour="yellow", ec='black', (w=2))
ax.add_patch(plt.Circle((0.35, -0.35), 0.12,
        color='black'))
ax.add_patch(plt.Circle(0.35,0.35), 0.12,
        color='black'))
smile_x=[-0.4, -0.2, 0.02, 0.4]
smile_y=[-0.2, 0.4, -0.45, 0.4, 0.2)
ax.plot(smile_x, smile_y, color='black',
        (w=3, solid_capstyle='round'))
ax.set_xline(-1.2, 1.2)
ax.set_yline(-1.2, 1.2)
ax.axis('off')
plt.show()
```

Result:- Thus the smiling face was drawn successfully



Scanned with OKEN Scanner

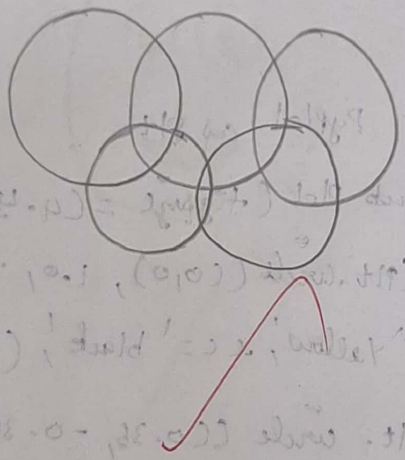
EXP-4 OLYMPIC RINGS using ellipse

Aim:

To implement the program to draw a olympic logo using ellipse

Code:

```
import matplotlib.pyplot as plt
import math
def ellipse(cx,cy,rx,ry):
t=[math.radians(i) for i in range(360)]
return [cx+rx*math.cos(a) for a in t],
    [cy+ry*math.sin(a) for a in t]
rings=[(2,4,'b'),(5,4,'k'),(8,4,'r'),
    (3.5,2.5,'y'),(6.5,2.5,'g')]
for x,y,c in rings:
    x,y=ellipse(x,y,1.5,.8)
    plt.plot(x,y,c,lw=3)
plt.axhline(0,color='k')
plt.axvline(0,color='k')
plt.grid()
plt.xlim(0,10)
plt.ylim(0,6)
plt.xticks(range(11))
plt.yticks(range(4))
```



```
plt.axis('equal')
plt.title("olympic logo - DDA")
plt.show()
```

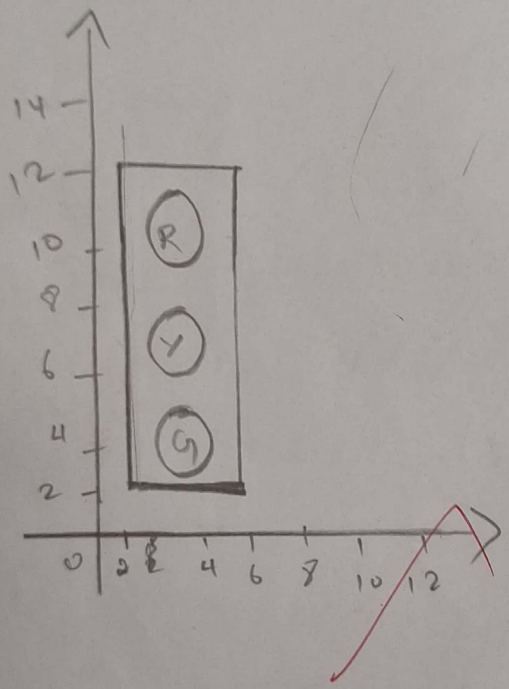
Result:- olympic logo using ellipse is done successfully

# EXP-08 Traffic signal using Bresenham's algorithm

AIM:- To Implement the program to draw the traffic signal using bresenham's algorithm

Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots()
box = Rectangle((2,1), 4, 10, facecolor="black")
ax.add_patch(box)
red = Circle((4,8), 1, facecolor="red"
             edgecolor="white")
ax.add_patch(red)
yellow = Circle((4,6), 1, facecolor="yellow",
                edgecolor="white")
ax.add_patch(yellow)
green = Circle((4,4), 1, facecolor="green",
               edgecolor="white")
ax.add_patch(green)
ax.set_xlim(0,8)
ax.set_ylim(0,12)
ax.grid(True)
```




Scanned with OKEN Scanner

```
plt.title("Traffic signal")
plt.show()
```

Result:-

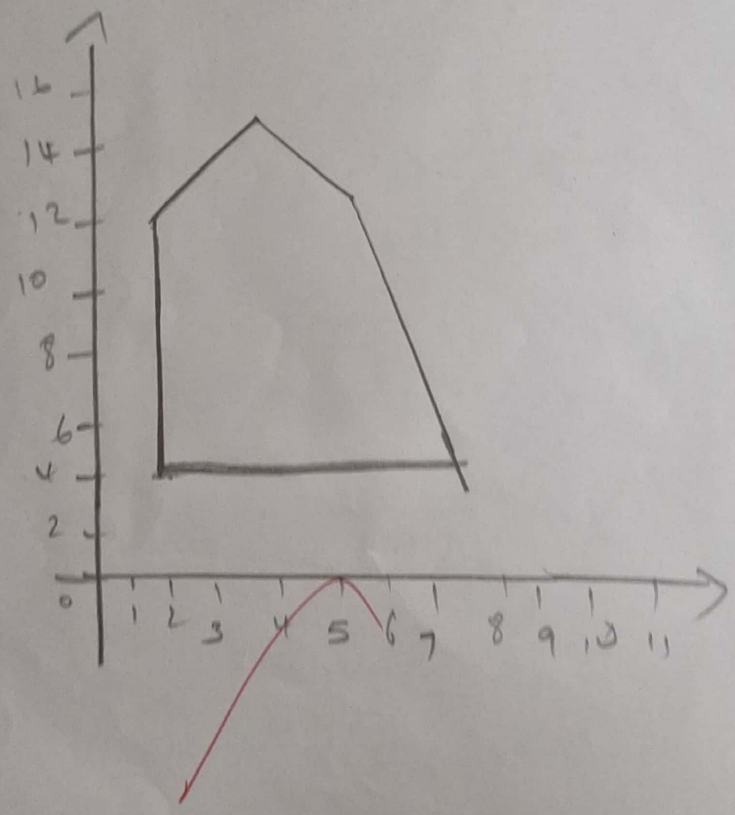
Hence the code is verified successfully

Lab-09 Implement scan line conversion of polygon & use flood fill algorithm

Aim: To implement a scan line conversion of polygon & use flood fill algorithm

Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import polygon, rectangle
Polygon = [(2,2), (8,2), (7,6), (5,8), (2,6)]
Colours = ('red', 'orange', 'yellow', 'green', 'blue',
           'purple', 'pink']
fig, ax = plt.subplots()
boundary = Polygon(Polygon,
                   closed = True,
                   fill = false,
                   edgecolor = 'black',
                   linewidth = 2)
ax.add_patch(boundary)
ymin = min(y for x, y in Polygon)
ymax = max(y for x, y in Polygon)
for y in range(ymin, ymax):
```




Scanned with OKEN Scanner

```
intersections = []
for i in range(len(polygon)):
    x1, y1 = polygon[i]
    x2, y2 = polygon[(i+1) % len(polygon)]
    if y1 != y2 and min(y1, y2) <= y < max(y1, y2):
        x = x1 + (y - y1) * (x2 - x1) / (y2 - y1)
        intersections.append(x)
intersections.sort()
colour = colours[ymin % len(colours)]
plt.xlim(0, 9); plt.ylim(0, 9)
plt.grid()
plt.title("
for i in range(0, len(intersections), 2):
    if i+1 < len(intersections):
        x_left = intersections[i]
        x_right = intersections[i+1]
        rect = plt.Rectangle((x_left, y),
                             x_right - x_left,
                             1,
                             facecolor=color
                             edgecolor='none')
        rect.set_clip_path(boundary)
        ax.add_patch(rect)
x = [p[0] for p in polygon] + [polygon[0][0]]
y = [p[1] for p in polygon] + [polygon[0][1]]
ax.plot(x, y, 'black', linewidth=2)
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
```

Scanned with OKEN Scanner

```
ax.set_xticks(range(11))
ax.set_yticks(range(11))
ax.set_aspect('equal')
ax.grid(True)
plt.title("Scan line conversion - Polygon Filling")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Result:

The program is implemented to scan line conversion of Polygon

Scanned with OKEN Scanner

Exp-10: Draw a basic shape & apply Transformations like rotation, translation & scaling

AIM:-

To implement a transformations to a basic shape

Code :-

```
import matplotlib.pyplot as plt
import numpy as np
P = np.array([[2,2],[5,2],[3,5],[2,2]])
t = P + [4,2]
s = P * 1.5
a = np.radians(45)
R = np.array([[np.cos(a), -np.sin(a)],
              [np.sin(a), np.cos(a)]])
r = P @ R.T
plt.plot(P[:,0], P[:,1], label="original")
plt.plot(t[:,0], t[:,1], label="translation")
plt.plot(s[:,0], s[:,1], label="scaling")
plt.plot(r[:,0], r[:,1], label="Rotation")
plt.axhline(0)
plt.axvline(0)
plt.grid()
plt.legend()
plt.show()
```

Result:-

Hence it is verified successfully

EXP-11 Draw a rectangle & apply Transformation

AIM:

To draw a rectangle & apply Transformation

Code:

```
import matplotlib.pyplot as plt
import numpy as np
P = np.array([[2,2], [6,2], [6,4], [2,4], [2,2]])
t = P + [4,2]
s = P * 1.5
a = np.radians(45)
R = np.array([[np.cos(a), -np.sin(a)],
              [np.sin(a), np.cos(a)]])
r = P @ R.T
plt.plot(P[:,0], P[:,1], label="original")
plt.plot(t[:,0], t[:,1], label="Translation")
plt.plot(s[:,0], s[:,1], label="Scaling")
plt.plot(r[:,0], r[:,1], label="Rotation")
plt.axvline(0)
plt.grid()
plt.legend()
plt.show()
```

Result:-

Hence, the code is verified successfully

EXP-12 Polygon clipping using sutherland hodgman

AIM

To clipping the polygon using sutherland hodgman

Code:

```
import matplotlib.pyplot as plt
P = [(1,1), (7,2), (6,6), (2,7)]
def clip (P, xmin, xmax, ymin, ymax):
return [(max(xmin, min(xmax, x)),
    max(ymin, min(ymax, y)) for x, y in P]
c = clip( P, 2, 6, 2, 6)
x, y = zip(*(P + [P[0]]))
plt.plot(x, y, label = "original")
x, y = zip((c + [c[0]]))
plt.plot( x, y, label = "clipped")
plt.grid ()
plt.legend ()
plt.show ()
```

Result :-

Hence the code is verified successfully

Scanned with OKEN Scanner

EXP-13 draw a 3D shape & apply transformation

AIM: To apply transformation on a 3D object

Code:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Poly3DCollection
import numpy as np
V = np.array([[0,0,0], [1,0,0], [1,1,0], [0,1,0],
              [0,0,1], [1,0,1], [1,1,1], [0,1,1]])
f = [[0,1,2,3], [4,5,6,7], [0,1,5,4],
     [2,3,7,6], [0,3,7,4], [1,2,6,5]]
a = np.radians(30)
R = np.array([[np.cos(a), -np.sin(a), 0],
              [np.sin(a), np.cos(a), 0], [0,0,1]])
cubes = [(V, 'blue', 'original'),
         (V + [3,0,0], 'red', 'Translation'),
         (V @ R + [0,3,0], 'green', 'rotation'),
         (V * 2 + [3,3,0], 'orange', 'scaling')]
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
for x, c, name in cubes:
    ax.add_collection3d(Poly3DCollection([x[i]
        for i in f], facecolors=c, edgecolors='black',
        alpha=.7))
```

```
p = z.mean(axis=0)
ax.text(*p, name)
ax.set_xlim(-1,6)
ax.set_ylim(-1,6)
ax.set_zlim(-1,6)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
plt.show()
```

Result :-

Hence the code is successfully verified

EXP 14 Rectangle using Bresenham's algorithm

Aim: To implement the rectangle using the Bresenham's algorithm in python

Code:

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, x1, y1):
    p = [];
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = 1
    if x0 < x1:
        sx = -1; sy = 1 if y0 < y1 else -1
    e = dx - dy
    while 1:
        p.append((x0, y0))
        if (x0, y0) == (x1, y1): return p
        e2 = 2 * e
        if e2 > -dy: e -= dy; x0 += sx
        if e2 < dx: e += dx; y0 += sy
    x, y, w, h = 2, 2, 8, 5
    xe, ye = x + w - 1, y + h - 1
    for a, b, c, d in [(x, y, xe, y), (x, ye, xe, ye),
                       (xe, y, xe, ye)]:
        p = bresenham(a, b, c, d)
```

Result

The rectangle is drawn successfully using Bresenham

Exp 15 Create and Render 3D objects using polygon and apply Basic color & shading Techniques

Aim:

To create and render 3D objects (such as cube or sphere) using Polygonal modeling and apply

* Flat shading
* Basic color techniques

Code:

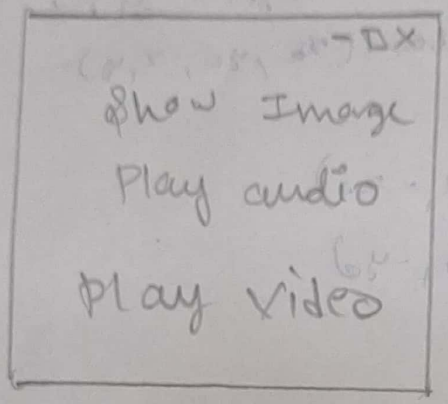
Importing:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],
                     [0,0,1],[1,0,1],[1,1,1],[0,1,1]])
faces = [[vertices[i] for i in [0,1,2,3]],
         [vertices[i] for i in [4,5,6,7]],
         [vertices[i] for i in [0,1,5,4]],
         [vertices[i] for i in [2,3,7,6]],
         [vertices[i] for i in [1,2,6,5]],
         [vertices[i] for i in [4,7,3,0]]]
```

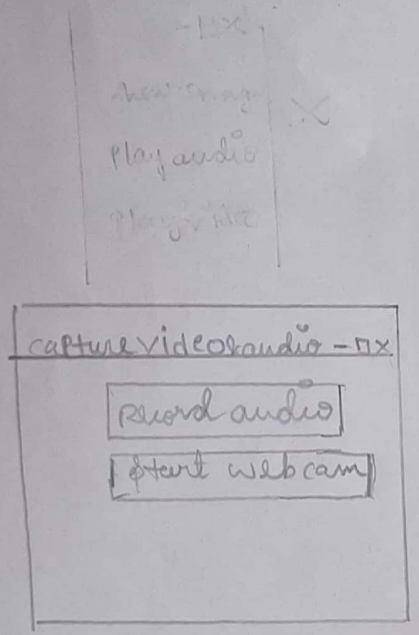
Result:

3D Polygon objects were successfully created and rendered. Basic color and flat shading techniques were applied to enhance visual representation

output



output




EX NO: 16 Create a multimedia application that integrates images, sound, and video in a simple user interface.

AIM: To develop a simple multimedia application that integrates.

* Images
* audio
* video

into a unified user interface using python and suitable libraries

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading

def play_audio():
    playsound('sample-audio.mp3') # Replace
with your file

def play_video():
    cap = cv2.VideoCapture('sample-video.mp4')
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
```

```
if cv2.waitKey(25) & 0xFF == ord('q'):
    break
cap.release()
cv2.destroyAllWindows()

def load_image():
img = Image.open('sample_image.jpg')
img = img.resize((300,300))
img_tk = ImageTk.PhotoImage(img)
Panel.configure(image = img_tk)
Panel.image = img_tk

# GUI setup
window = tk.Tk()
window.title("multimedia APP")

btn_img = tk.Button(window, text="show image",
btn_audio = tk.Button(window, text="Play audio",
threading.Thread(target = Play_audio).start())
btn_video = tk.Button(window, text="Play Video",
command = lambda: threading.Thread(target =
Play())

Panel = tk.Label(window)
Panel.pack()
btn.image.pack(pady=5)
btn_audio.pack(pady=5)
btn_video.pack(pady=5)
window.mainloop()
```

Result: A multimedia app was successfully created

Scanned with OKEN Scanner

EXP NO:17 Capture video/audio from webcam or microphone and display on multimedia interface

AIM: to develop a program that captures

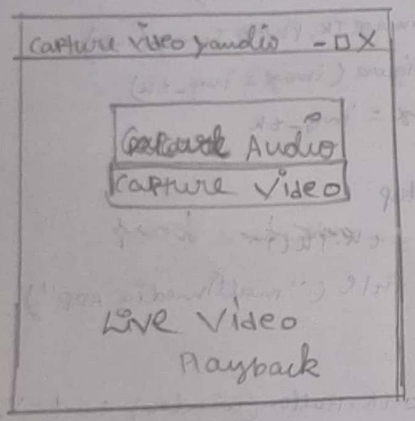
Program

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk

# audio recording function
def record_audio():
duration = 5 # seconds
fs = 44100 # sampling rate
Print("Recording audio")
audio = sd.rec(int(duration * fs), samplerate
  = fs, channels=2)
sd.wait() # wait until recording is finished
Print("Audio recording saved.")

# Video Capture function
def capture_video():
  cap = cv2.VideoCapture(0)
  Print("Press 'q' to stop video.")
```

output




Scanned with OKEN Scanner

```
# GUI setup
window = tk.Tk()
window.title("Capture audio & video")

btn_audio = tk.Button(window, text="Record audio", command=lambda: threading.Thread(target=record_audio).start())

btn_video = tk.Button(window, text="Start webcam", command=lambda: threading.Thread(target=capture_video).start())

btn_audio.pack(pady=10)
btn_video.pack(pady=10)
window.mainloop()
```

Result: Live video & audio were successfully captured using webcam & microphone & displayed / saved through a multimedia interface

EXP NO: 18 Create an application to send receive multimedia message (Text, image, audio)

AIM: TO develop a basic chat like application that allow:

* Sending & receiving text, image & audio
* Using socket like selection and simulated messaging GUI

Program:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading

def send_text():
    msg = text_entry.get()
    if msg:
        chat_box.insert(tk.END, f"You: {msg}\n")
        text_entry.delete(0, tk.END)

def send_image():
    path = filedialog.askopenfilename(filetypes=
[("image files", "*.jpg *.png *.jpeg")])
```

```
img.thumbnail((100,100))
img_tk=imageTk.PhotoImage(img)
image_label=tk.Label(chat_frame, image=img_tk)
image_label.image=img_tk
image_label.pack()
chat_box.yview(tk.END)
```

```
def send_audio():
    Path=filedialog.askopenfilename(filetypes=
["audio File","*.mp3 *.wav"])
    if path:
        chat_box.insert(tk.END,f"You sendan
audio message:{path}\n")
        Threading.Thread(target=lambda:playsound
(Path)).start()
```

Result:

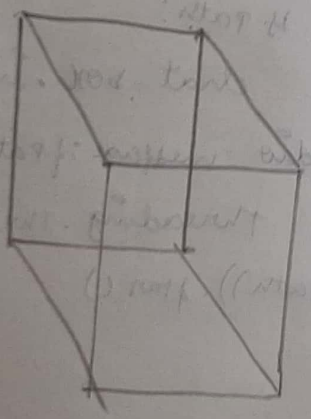
A multimedia messaging interface was successfully created with options to send text, image & audio content

EXP NO: 19 Create a 3D model using Blender & apply Basic shading & texture

AIM:

To create a simple 3D model (eg. a house or object) using Blender, and apply

* Basic shading
* textures (image based & procedural)



Procedure:

1) Launch Blender.
2) model creation: * use add > mesh to
   * use add > mesh to insert basic shape like cube, cylinder, plane, etc
   * combine these objects to form a house or similar structure
   * use edit mode (Tab key) for external and shaping
3) shading:
   * switch to the shading workspace
   * add a new material for each objects
   * adjust Base color, specular & Roughness
4) Texture mapping:
   * In the shader editor, add an image texture node.

* Load an external image
* Connect it to the Base Color of the Principled BSDF shader
* use UV editing to align the texture

5. Lighting and Camera:

* add light source (Sun or Point light)
* Position the camera to frame the model

6. Render Settings

* choose cycle or eevee rendering engine
* adjust resolution & sampling

Result:

* 3D object (house) was successfully modeled in Blender